Chapter 2: Intro to Relational Model

Database System Concepts, 7th Ed.

Outline

- Structure of Relational Databases
- Database Schema
- Keys
- Schema Diagrams
- Relational Query Languages
- The Relational Algebra

Relational Model Concepts

- The relational Model of Data is based on the concept of a Relation.

- A Relation is a mathematical concept based on the ideas of sets.
- The model was first proposed by Dr. E.F. Codd of IBM in 1970 in the following paper:
  - "A Relational Model for Large Shared Data Banks"

A relational model represents the database as a collection of relations.

Formal relational model terminology:

Relation: table.

Tuple: individual row in a relation

Attribute: column header or column name

Example of a *Instructor* Relation

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

attributes (or columns)

tuples (or rows)

Relation Schema and Instance

- $A_1, A_2, \ldots, A_n$ are *attributes*
- $R = (A_1, A_2, \ldots, A_n)$ is a *relation schema*

  Example:

  ***instructor* = (*ID*, *name*, *dept_name*, *salary*)**

- A relation instance *r* defined over schema *R* is denoted by *r* (*R*).
- The current values a relation are specified by a table
- An element ***t*** of relation ***r*** is called a *tuple* and is represented by a *row* in a table

Attributes

- The set of allowed values for each attribute is called the **domain** of the attribute
  - The domain of Address is set of all valid addresses , which is specified as character array.
  - Domain of age is integer numbers between 18-75
- **Degree (arity)** of a relation is number of attributes n of its relation.

Ex: STUDENT(name,usn,dob) here degree 3.

- Attribute values are (normally) required to be **atomic**; that is, indivisible
- The special value ***null*** is a member of every domain. Indicated that the value is "unknown"
- The null value causes complications in the definition of many operations

Relations are Unordered

- Order of tuples is irrelevant (tuples may be stored in an arbitrary order)
- Example: *instructor* relation with unordered tuples

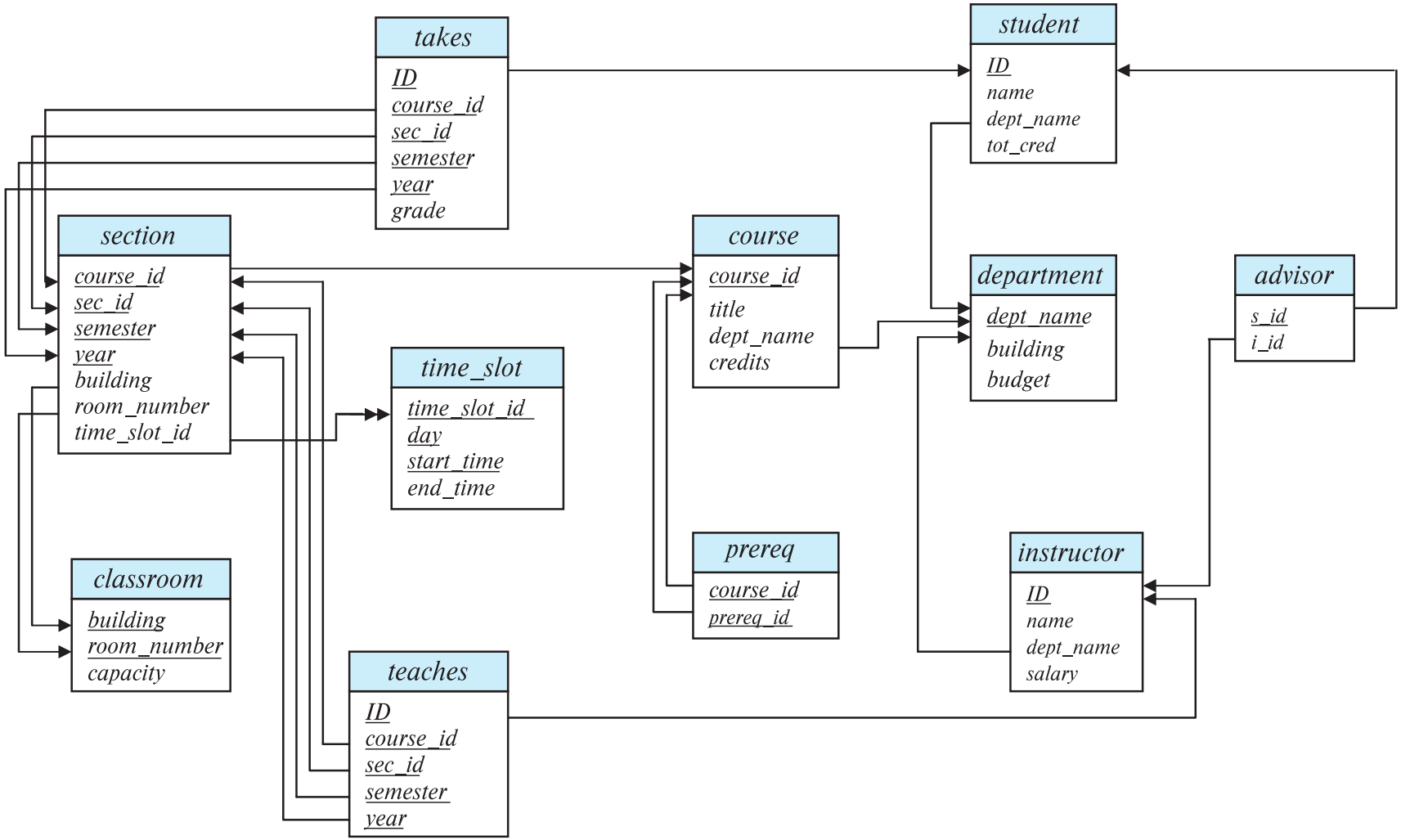
ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Database Schema

- Database schema -- is the logical structure of the database.
- Database instance -- is a snapshot of the data in the database at a given instant in time.
- Example:
  - schema: i*nstructor* (*ID, name, dept_name, salary*)
  - Instance:

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Schema Diagram for University Database



Keys

- Key specifies that no two tuples can have the same combination of values for all their attributes.
- Super key: **Is a set of attributes K of a relation schema R which specifies uniqueness constraint that no two distinct tuples in any state r(R) can have same combination of values for attributes in the set K.**
- *K* is a **superkey** of *R* if values for *K* are sufficient to identify a unique tuple of each possible relation *r(R)*
  - Example: {*ID*} and {ID,name} are both superkeys of *instructor.*

Ex: STUDENT(usn,name,age,address)

{usn,age,name} as superkey.

Remove *age* from the set still it is a superkey.

STUD_NO	SNAME	ADDRESS	PHONE
1	Tom	New York	123456789
2	John	Los Angeles	223365796
3	Alice	Chicago	175468965

A relation may have more than one key . In this case each key is called as **candidate key**.

- Example: **STUDENT(usn,name email,address,dob)**

Here usn and email are candidate keys.

{*ID*} is a candidate key for *Instructor*

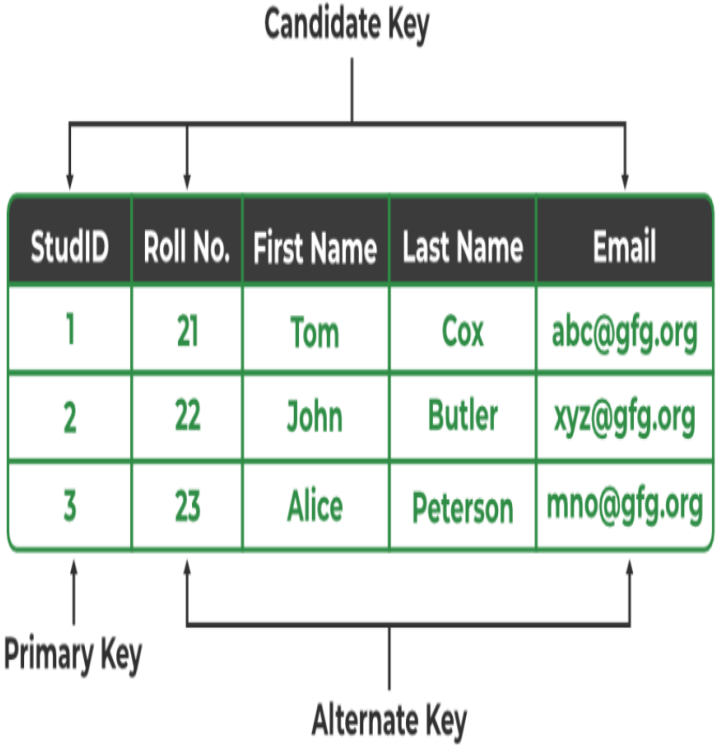
One of the candidate key is designated as **primary key** of the relation.

A primary key is a candidate key that can be used to identify uniquely each tuple in the relation.

constraints on NULL values:

Specifies whether NULL values are permitted or not.

For ex: **name cannot be NULL**



Foreign key constraint

Foreign key constraint:

- Used to specify a **relationship** among tuples in two relations:
- Informally, it states that a tuple in one relation that refers to another relation must refer to an existing tuple in other relation
- Value in one relation must appear in another
- **Referencing** relation
- **Referenced** relation
- Example: *dept_name* in i*nstructor* is a foreign key from *instructor* referencing *department*

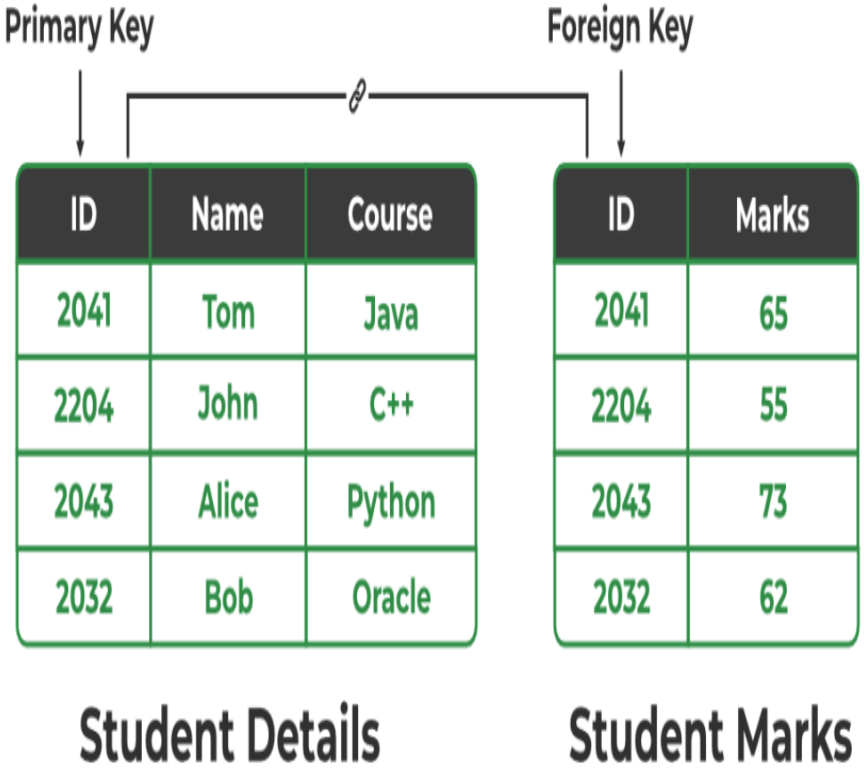
Foreign key:

A set of attribute FK in relation schema R2 is a foreign key of R2 that references R1 if it satisfies the following rules.

1) The attributes in FK have the same domain(s) as the primary key attributes PK of R1;
2) a value of FK in a tuple t1 of the current state r1(R1) either occurs as a value of PK for some tuple t2 in the current state r2(R1) or is NULL.

The attributes in FK are said to reference or refer to relation R1.

If t2[FK]=t1[PK] , then tuple t2 references of refer to tuple t1.



Relational Query Languages

- **Procedural versus non-procedural, or declarative**
- "Pure" languages:
  - **Relational algebra**
  - Tuple relational calculus
  - Domain relational calculus
- The above 3 pure languages are equivalent in computing power
- We will concentrate in this chapter on relational algebra
  - Not Turing-machine equivalent
- Six basic operators
  - select: σ
  - project: Π
  - union: $\cup$
  - set difference: –
  - Cartesian product: x
  - rename: ρ

Relational Algebra

- **The basic set of operations** for the relational model is known as the **relational algebra.**

- The **algebra operations** thus produce new relations, which can be further manipulated using operations of the same algebra.

- A sequence of relational algebra operations forms a **relational algebra expression.**

  – The result of a relational algebra expression is also a relation that represents the result of a database query (or retrieval request).

Select Operation

- The **select** operation selects tuples that satisfy a given predicate.
- Notation: $\sigma_p(r)$
- p is called the **selection predicate**
- Example: **select those tuples of the *instructor* relation where the instructor is in the "Physics" department.**
  - Query

    $\sigma_{dept\_name="Physics"}(instructor)$

  - Result

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Select Operation (Cont.)

- We allow comparisons using

  $=, \neq, >, \geq. <. \leq$

  in the selection predicate.
- We can **combine several predicates into a larger predicate** by using the connectives:

  $\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**)
- Example: Find the instructors in Physics with a salary greater $90,000, we write:

  $$\sigma_{dept\_name=\text{“Physics”} \wedge salary > 90,000}(instructor)$$
- The select predicate may include comparisons between two attributes.
  - Example**, find all departments whose name is the same as their building name:**
  - $\sigma_{dept\_name=building}(department)$

Project Operation

- A unary operation that returns its argument relation, with certain attributes left out.
- Notation:

$$\Pi_{A_1,A_2,A_3 \ldots A_k}(r)$$

where A_1, A_2, …, A_k are attribute names and r is a relation name.

- The result is defined as the relation of k columns obtained by erasing the columns that are not listed
- Duplicate rows removed from result, since relations are sets

Project Operation Example

- Example: eliminate the *dept_name* attribute of *instructor*
- Query:

$$\Pi_{ID,\ name,\ salary}\ (instructor)$$

- Result:

ID	*name*	*salary*
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

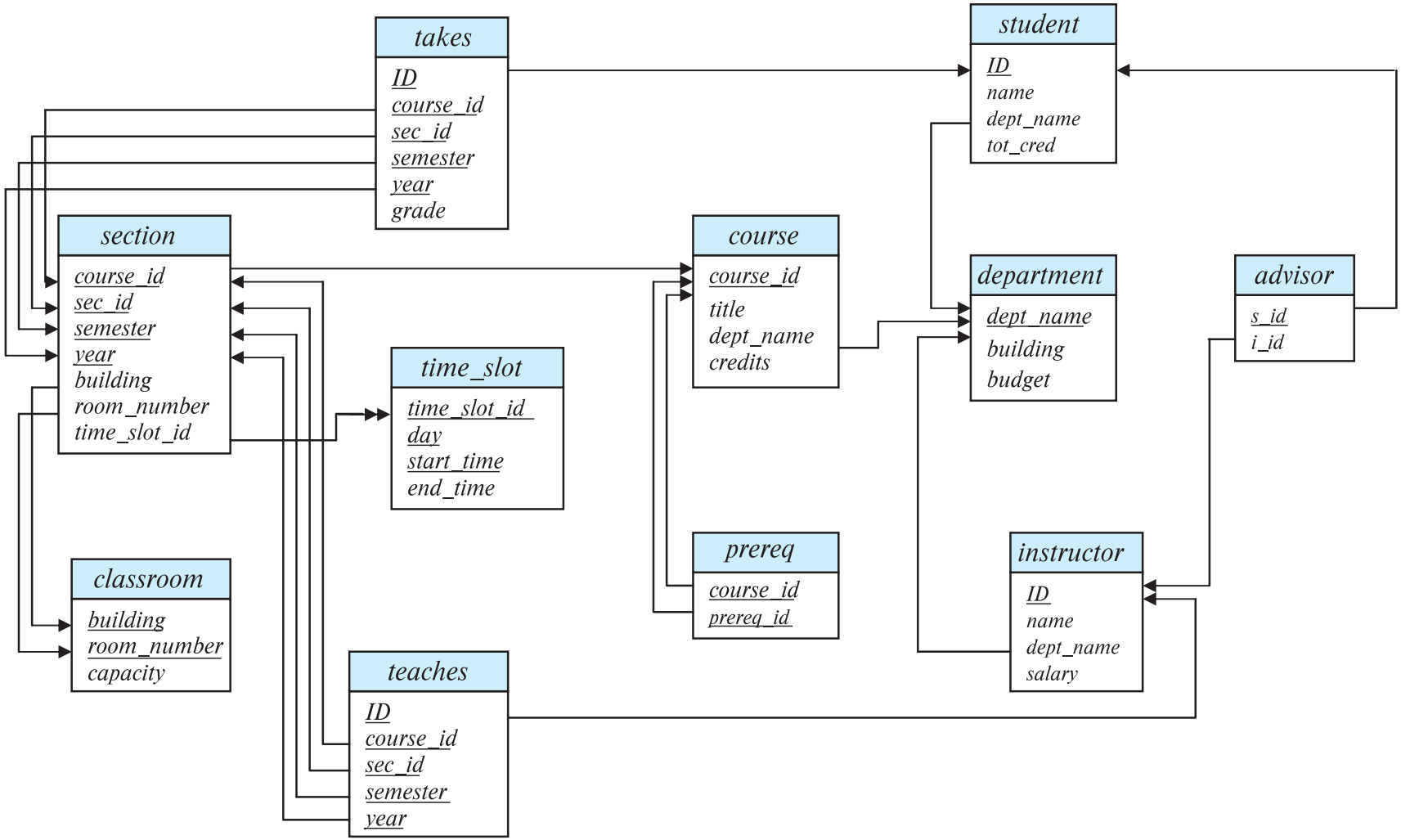
Composition of Relational Operations

- The result of a relational-algebra operation is relation and therefore of relational-algebra operations can be composed together into a **relational-algebra expression**.
- Consider the query -- **Find the names of all instructors in the Physics department.**

$$\Pi_{name}(\sigma_{dept\_name = \text{“Physics”}} (instructor))$$

- Instead of giving the name of a relation as the argument of the projection operation, we give an expression that evaluates to a relation.

Schema Diagram for University Database



Instructor

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Teaches

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

Student

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

Section

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

Takes

ID	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-

Course

course_id	title	dept_name	credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

Cartesian-Product Operation

- The Cartesian-product operation (denoted by X) allows us to combine information from any two relations.
- Operation produces new relation by combining **every row from one table with every row from another table. — everything is paired with everything**.

Table A (1 x 3)

ProductName
Laptop
Smartphone
Tablet

Table B (1 x 3)

RegionName
North America
Europe
Asia

Result Set (2 x 9)

ProductName	RegionName
Laptop	North America
Laptop	Europe
Laptop	Asia
Smartphone	North America
Smartphone	Europe
Smartphone	Asia
Tablet	North America
Tablet	Europe
Tablet	Asia

Cartesian-Product Operation

SNO	FNAME	LNAME
1	Albert	Smith
2	Pearl	Weber

ROLLNO	AGE
5	18
9	21

SNO	FNAME	LNAME	ROLLNO	AGE
1	Albert	Smith	5	18
1	Albert	Smith	9	21
2	Pearl	Weber	5	18
2	Pearl	Weber	9	21

Cartesian Product: $R \times S$

R

A	B	C
a1	b1	c3
a2	b1	c5
a3	b4	c7

S

E	F
e1	f1
e2	f5

$R \times S$

A	B	C	E	F
a1	b1	c3	e1	f1
a1	b1	c3	e2	f5
a2	b1	c5	e1	f1
a2	b1	c5	e2	f5
a3	b4	c7	e1	f1
a3	b4	c7	e2	f5

Cartesian-Product Operation

- The Cartesian product combines every row of one table with every row of another table, producing all the possible combination.
- Example: the Cartesian product of the relations *instructor* and t*eaches* is written as:

  instructor X *teaches*

  pairs **every instructor** with **every course record**, whether they belong together or not.
- We construct a tuple of the result out of each possible pair of tuples: one from the *instructor* relation and one from the *teaches* relation (see next slide)
- Cartesian product pairs every tuple of one relation with every tuple of another, and we use relation names to distinguish attributes with the same name.
- Since the **instructor *ID* appears in both relations** we distinguish between these attribute by attaching to the attribute the name of the relation from which the attribute originally came.
  - ***instructor.ID***
  - ***teaches.ID***

The *instructor* x *teaches* table

In general, the **result of R(A1, A2, . . ., An) x S(B1, B2, . . ., Bm)** is a relation Q with degree n + m attributes Q(A1, A2, . . ., An, B1, B2, . . ., Bm), in that order.

This creates **many meaningless rows** — instructors paired with courses they never taught

instructor.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2017
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2018
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2018
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
15151	Mozart	Music	40000	10101	CS-101	1	Fall	2017
15151	Mozart	Music	40000	10101	CS-315	1	Spring	2018
15151	Mozart	Music	40000	10101	CS-347	1	Fall	2017
15151	Mozart	Music	40000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
15151	Mozart	Music	40000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
22222	Einstein	Physics	95000	10101	CS-101	1	Fall	2017
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2018
22222	Einstein	Physics	95000	10101	CS-347	1	Fall	2017
22222	Einstein	Physics	95000	12121	FIN-201	1	Spring	2018
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

Join Operation

- **The Cartesian-Product**

  instructor X teaches

  associates every tuple of instructor with every tuple of teaches.

  - **Most of the resulting rows have information about instructors who did NOT teach a particular course -** An instructor might appear with a course they never taught
  - **Match instructors with the courses they actually taught.**

- To get only those tuples of "*instructor* X *teaches* " that pertain to instructors and the courses that they taught, we write:

  $$\sigma_{instructor.id = teaches.id} (instructor \times teaches))$$

  - We get only those tuples of "***instructor X teaches***" that pertain to instructors and the courses that they taught.

- The result of this expression, shown in the next slide

Join Operation (Cont.)

- The table corresponding to:

$$\sigma_{instructor.id = teaches.id}(instructor \times teaches))$$

instructor.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
32343	El Said	History	60000	32343	HIS-351	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-101	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-319	1	Spring	2018
76766	Crick	Biology	72000	76766	BIO-101	1	Summer	2017
76766	Crick	Biology	72000	76766	BIO-301	1	Summer	2018
83821	Brandt	Comp. Sci.	92000	83821	CS-190	1	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-190	2	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-319	2	Spring	2018
98345	Kim	Elec. Eng.	80000	98345	EE-181	1	Spring	2017

Join Operation (Cont.)

- **The join operation allows us to combine a select operation and a Cartesian-Product operation into a single operation.**
- Consider relations ***r*** **(*R*) and *s* (*S*)**
- Let "theta" be **a predicate on attributes** in the schema R "union" S. The join operation $\mathbf{r} \bowtie_{\theta} \mathbf{s}$ is defined as follows:

$$r \bowtie_{\theta} s = \sigma_{\theta} (r \times s)$$

- Thus

$$\sigma_{instructor.id = teaches.id} (instructor \times teaches))$$

- Can equivalently be written as

$$instructor \bowtie_{Instructor.id = teaches.id} teaches.$$

The result contains **only instructors and the courses they taught**

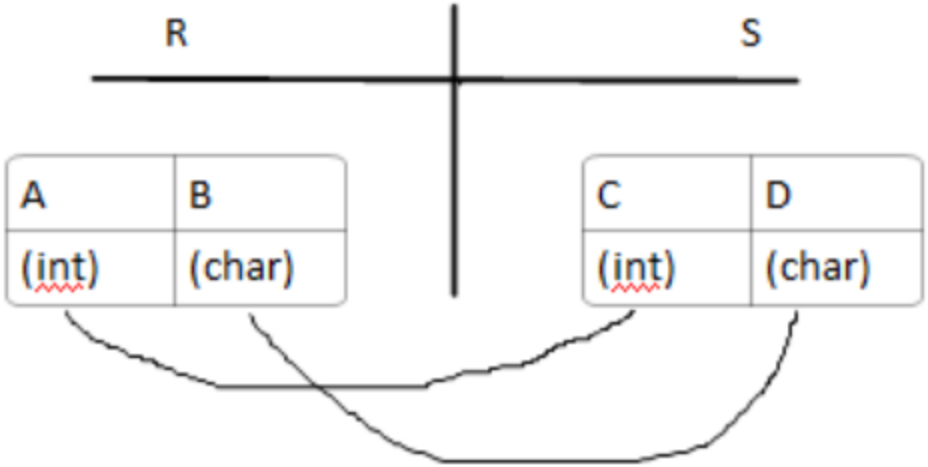
A join is a Cartesian product followed by a selection that keeps only matching rows

Union Operation

- The union operation allows us to combine two relations
- Notation: $r \cup s$
- Union gives us **all tuples that appear in either relation, or in both**.
- **For $r \cup s$ to be valid.**

  1. *r, s* must have the *same* **arity** (same number of attributes)
  2. The attribute domains must be **compatible**

  (example: 2nd column of *r* deals with the same type of values as does the 2nd column of *s*)



Relational Algebra Operations From Set Theory (contd)

- **UNION Example** **STUDENT∪INSTRUCTOR**

STUDENT	FN	LN
	Susan	Yao
	Ramesh	Shah
	Johnny	Kohler
	Barbara	Jones
	Amy	Ford
	Jimmy	Wang
	Ernest	Gilbert

INSTRUCTOR	FNAME	LNAME
	John	Smith
	Ricardo	Browne
	Susan	Yao
	Francis	Johnson
	Ramesh	Shah

(b)

FN	LN
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert
John	Smith
Ricardo	Browne
Francis	Johnson

Student U instructor

Union Operation

- Example: to find all courses taught in the **Fall 2017 semester**, or in the **Spring 2018 semester**, or **in both**

  Courses taught in Fall 2017

- $\Pi_{course\_id}(\sigma_{semester="Fall" \wedge year=2017}(section))$ -→ A list of course IDs taught in **Fall 2017**

  Courses taught in Spring 2018

- $\Pi_{course\_id}(\sigma_{semester="Spring" \wedge year=2018}(section))$ → A list of course IDs taught in **Spring 2018**

Union Operation (Cont.)

- Result of: If a course was taught in **both semesters**, it appears **only once**.

$$\Pi_{course\_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(section)) \cup \Pi_{course\_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(section))$$

course_id
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101

Relational Algebra Operations From Set Theory (cont.)

- **INTERSECTION OPERATION**

The result of this operation, denoted by R ∩ S, is a relation that includes all tuples that are in both R and S. The two operands must be "type compatible"

Example: The result of the intersection operation (figure below) includes only those who are both students and instructors.

FN	LN
Susan	Yao
Ramesh	Shah

(d)

FN	LN
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

STUDENT ∩ INSTRUCTOR

Set-Intersection Operation

- The intersection operation finds only the tuples that appear in both relations.
- The set-intersection operation allows us to find tuples that are in both the input relations.
- Notation: $r \cap s$
- Assume:
  - ***r*, *s* have the *same arity***
  - **attributes of *r* and *s* are compatible**
  - Intersection returns only those tuples that exist in both relations
  - Example: **Find the set of all courses taught in both the Fall 2017 and the Spring 2018 semesters.**

$$\Pi_{course\_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(section)) \cap \Pi_{course\_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(section))$$

  - Result: The result is the set of courses that were taught in both semesters.

course_id
CS-101

Relational Algebra Operations From Set Theory (cont.)

- **Set Difference (or MINUS) Operation**

The result of this operation, denoted by R - S, is a relation that includes all tuples that are in R but not in S. The two operands must be "type compatible".

Example: The figure shows the names of students who are not instructors, and the names of instructors who are not students.

FN	LN
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

STUDENT-INSTRUCTOR

FNAME	LNAME
John	Smith
Ricardo	Browne
Francis	Johnson

INSTRUCTOR-STUDENT

Set Difference Operation

- **The set-difference operation allows us to find tuples that are in one relation but are not in another.**
- Notation $r - s$
  - **Difference = A minus B**
  - **Or IN A AND NOT IN B**
- Set differences must be taken between **compatible** relations.
  - r and s must have the same arity
  - attribute domains of r and s must be compatible
- Example: to find all courses taught in the Fall 2017 semester, but not in the Spring 2018 semester

$$\Pi_{course\_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(section)) -$$
$$\Pi_{course\_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(section))$$

course_id
CS-347
PHY-101

The Assignment Operation

- The assignment operation lets us **break a complex query into smaller, readable steps**
- It is convenient at times to write a relational-algebra expression by assigning parts of it to temporary relation variables.
- The assignment operation is denoted by ← **and works like assignment in a programming language.**
- Example: **Find all instructor in the "Physics" and Music department.**

$$Physics \leftarrow \sigma_{dept\_name="Physics"}(instructor)$$

$$Music \leftarrow \sigma_{dept\_name="Music"}(instructor)$$

$$Physics \cup Music$$

This gives all instructors who are in either Physics or Music.

- With the assignment operation, a query can be written as a sequential program consisting of a series of assignments followed by an expression whose value is displayed as the result of the query.
- With assignment, the query becomes **easier to read, easier to understand, and easier to debug**. **The assignment operation lets us store intermediate results and write relational-algebra queries step by step.**

The Rename Operation

- The **results of relational-algebra expressions(a relation) do not have a name** that we can use to refer to them.
- The rename operator, ρ, is provided for that purpose
- The expression:

$$\rho_x(E)$$

returns the result of expression *E* under the name *x*

The data does not change — only the **name** changes.

Suppose E returns **a table of instructors** in the **Physics department**.

Using rename, we can call this table **Physics_Instructors**

- Another form of the rename operation:

$$\rho_{x(A1,A2,\ldots An)}(E)$$

- Renames the **relation** to x
- Renames the **attributes (columns)** to A1, A2, …, An"

The number of attribute names must match the number of attributes in E.

Equivalent Queries

- **There is more than one way to write a query in relational algebra.**
- Example: Find **information about courses taught by instructors in the Physics department with salary greater than 90,000**
- Query 1

$$\sigma_{dept\_name=\text{“Physics”} \wedge salary > 90{,}000}(instructor)$$

- Query 2

$$\sigma_{dept\_name=\text{“Physics”}}(\sigma_{salary > 90.000}(instructor))$$

- The two queries are not identical; they are, however, equivalent -- they give the same result on any database.
- Equivalent **relational-algebra expressions may look different, but always return the same result**
- Database systems rewrite queries into equivalent forms that run faster - optimize queries

Equivalent Queries

- In relational algebra, **the same query result can be written in different ways**

- Example: **Find information about courses taught by instructors in the Physics department**
- Query 1:
  - First join EVERYTHING → then filter Physics.

$$\sigma_{dept\_name=\text{“Physics”}}(instructor \bowtie_{instructor.ID = teaches.ID} teaches)$$

- Query 2
  - First filter Physics instructors → then join

$$(\sigma_{dept\_name=\text{“Physics”}}(instructor)) \bowtie_{instructor.ID = teaches.ID} teaches$$

- The two queries are not identical; they are, however, equivalent -- they give the same result on any database.
- Query 2 is preferred because we reduce the table first (Physics instructors only), so the join becomes faster.

End of Chapter 2

Problems on Relational Algebra

Consider the following schema:

SAILORS(sid:integer,sname:string, rating:integer, age:real)

BOATS(bid:integer, bname:string, color:string)

RESERVES(sid:integer,bid:integer,day:date)

Write relational algebra expressions for the following

1. Find the names of sailors who have reserved boat 103
2. Find the names of sailors who have reserved a red boat
3. Find the colors of boats reserved by ‘john’
4. Find the names of sailors who have reserved at least one boat
5. find the names of sailors who have reserved a red or green boat
6. Find the names of sailors who have reserved a red boat and a green boat
7. Find the names of sailors who have reserved at least two boats
8. Find the sids of sailors with age over 20 who have not reserved a red boat

Problems on Relational Algebra

1) Names of sailors who have reserved boat 103

$$\pi_{sname}(SAILORS \bowtie \sigma_{bid=103}(RESERVES))$$

2) Names of sailors who have reserved a red boat

$$\pi_{sname}(SAILORS \bowtie RESERVES \bowtie \sigma_{color='red'}(BOATS))$$

3) Colors of boats reserved by 'john'

$$\pi_{color}(\sigma_{sname='john'}(SAILORS) \bowtie RESERVES \bowtie BOATS)$$

4) Names of sailors who have reserved at least one boat

$$\pi_{sname}(SAILORS \bowtie RESERVES)$$

5) Names of sailors who have reserved a red or green boat

$$\boldsymbol{\pi_{sname}\left(SAILORS \bowtie RESERVES \bowtie \sigma_{color='red' \vee color='green'}(BOATS)\right)}$$

(or using union)

$$\boldsymbol{\pi_{sname}(SAILORS \bowtie RESERVES \bowtie \sigma_{color='red'}(BOATS))}$$
$$\boldsymbol{\cup\ \pi_{sname}\left(SAILORS \bowtie RESERVES \bowtie \sigma_{color='green'}(BOATS)\right)}$$

6) Names of sailors who have reserved a red boat AND a green boat

$$R = \pi_{sid}(RESERVES \bowtie \sigma_{color='red'}(BOATS))$$

$$G = \pi_{sid}(RESERVES \bowtie \sigma_{color='green'}(BOATS))$$

$$\pi_{sname}\left(SAILORS \bowtie (R \cap G)\right)$$

7) Names of sailors who have reserved at least two boats

Self-join RESERVES on same sid but different bid:

$$\pi_{sname}(SAILORS \bowtie \pi_{sid}(\sigma_{R1.sid=R2.sid \wedge R1.bid \neq R2.bid}(\rho_{R1}(RESERVES) \times \rho_{R2}(RESERVES))))$$

(sid=22, bid=101) with (sid=22, bid=103)
(sid=22, bid=103) with (sid=22, bid=101)

8) SIDs of sailors with age > 20 who have NOT reserved a red boat

$$A = \pi_{sid}(\sigma_{age>20}(SAILORS))$$

$$B = \pi_{sid}(RESERVES \bowtie \sigma_{color='red'}(BOATS))$$

$$A - B$$

Problems on Relational Algebra

Given relations:

- **employee**(**person-name**, street, city)
- **works**(**person-name, company-name,** salary)
- **company**(**company-name**, city)

1. Find employees who earn a **salary greater than 60,000**.
2. Find the **distinct cities** where employees live.
3. Find all possible combinations of **employees and companies**.
4. Find employee names along with the **company they work for**.
5. Find employees who work in companies located in the **same city** they live in.
6. Find all **cities** where either employees live **or** companies are located.
7. Find cities where **both employees live and companies are located**.
8. Find employees who **do not live in the same city as their company**.
9. Find employees who **do not work for any company**.